

# Perceiver — Sintesi Teorica (v2)

Versione con “sei qui” sull’architettura

Questo documento è una sintesi della **Sezione 1 (*Perceiver*)** degli appunti completi (`appunti_ml.tex`, pp. 5–87). Raccoglie **la teoria** dell’architettura — l’idea di fondo, i blocchi, il forward pass *passo per passo* con le forme delle matrici e gli esempi numerici, il *perché* di ogni scelta. È escluso solo il backward pass (le derivazioni dei gradienti: puro calcolo, non teoria).

**Come usare questo documento.** In cima a ogni pagina trovi la **striscia-pipeline**: le 7 fasi del Perceiver, con evidenziata in blu quella di cui tratta la pagina. A inizio di ogni sezione un box “**In breve**” ne ricapitola il contenuto. I box **verdi** sono le idee-chiave da memorizzare, i box **verde acqua** esempi numerici svolti.

## Il Perceiver in 60 secondi

Il **Perceiver** (Jaegle et al., 2021) è un’architettura **general-purpose**: processa qualsiasi modalità (immagini, audio, video, point cloud) con un *unico* modello, senza componenti specifici per il dominio. Risolve il collo di bottiglia quadratico  $O(M^2)$  della self-attention introducendo un **latent array** di dimensione fissa  $N \ll M$ : una **cross-attention** comprime l’input enorme in questo spazio latente (costo  $O(MN)$ ), poi un **latent transformer** lo raffina con self-attention profonda (costo  $O(N^2)$ ). Il blocco viene ripetuto  $T$  volte con **weight sharing**  $\Rightarrow$  il modello è di fatto un **RNN** che raffina progressivamente i latenti rileggendo lo stesso input.

Input  $\rightarrow$  Byte Array + Fourier PE  $\rightarrow$  [Cross-Att  $\rightarrow$  Latent Transformer]  $\times T \rightarrow$   
Pooling  $\rightarrow$  Logits

## Indice

|          |                                                          |          |
|----------|----------------------------------------------------------|----------|
| <b>1</b> | <b>Il problema: la complessità quadratica</b>            | <b>2</b> |
| <b>2</b> | <b>L’idea centrale: dalla self- alla cross-attention</b> | <b>3</b> |
| <b>3</b> | <b>Architettura in tre stadi</b>                         | <b>4</b> |
| <b>4</b> | <b>Il forward pass passo per passo</b>                   | <b>6</b> |
| 4.1      | Input: il byte array . . . . .                           | 6        |
| 4.2      | Positional encoding: le Fourier features . . . . .       | 7        |
| 4.3      | Il latent array . . . . .                                | 8        |
| 4.4      | Il blocco di cross-attention . . . . .                   | 9        |
| 4.5      | Il latent transformer . . . . .                          | 12       |
| 4.6      | Weight sharing e iterazioni . . . . .                    | 14       |
| 4.7      | Output: pooling e classificazione . . . . .              | 15       |

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| <b>5</b> | <b>I componenti ricorrenti</b>                                  | <b>17</b> |
| 5.1      | LayerNorm — normalizzazione . . . . .                           | 17        |
| 5.2      | GELU — la funzione di attivazione . . . . .                     | 17        |
| 5.3      | Residual connection — la somma che salva il gradiente . . . . . | 18        |
| 5.4      | Softmax — da punteggi a probabilità . . . . .                   | 18        |
| <b>6</b> | <b>Dettagli di training (ImageNet)</b>                          | <b>19</b> |
| <b>7</b> | <b>Risultati chiave</b>                                         | <b>19</b> |
| <b>8</b> | <b>Ablation: cosa conta davvero</b>                             | <b>20</b> |
|          | <b>Riferimenti</b>                                              | <b>21</b> |

# 1 Il problema: la complessità quadratica

## In breve

**CNN**: scalabili ( $O(MKL)$ ) ma servono architetture specializzate per ogni dominio. **Transformer**: generali ma costano  $O(M^2)$ . **Perceiver**: generale e scalabile, grazie a un **latent bottleneck**  $N \ll M$  — la complessità passa da  $O(M^2)$  a  $O(MN) + O(N^2)$ .

Prima del Perceiver, due famiglie di reti dominavano, ognuna con un pregio e un difetto opposti:

- **CNN** — *scalabili* (costo  $O(MKL)$ , lineare nel numero di input  $M$ ) ma richiedono un'**architettura specializzata per ogni dominio** (convoluzioni 2D per le immagini, 1D per l'audio, ...): poco generali.
- **Transformer** — *generali* (un solo meccanismo, l'attention, per qualsiasi dato) ma la self-attention costa  $O(M^2)$ : ogni elemento confronta sé stesso con tutti gli altri. Su input grandi diventa impraticabile — un'immagine ImageNet  $224 \times 224$  ha  $M = 50\,176$  pixel, cioè una matrice di attenzione da  $\sim 2.5$  miliardi di entry.

Il Perceiver **unisce il meglio dei due**: la *generalità* del Transformer e la *scalabilità* delle CNN. La chiave è un **latent bottleneck** — un collo di bottiglia di dimensione fissa attraverso cui far passare l'input.

| Famiglia         | Costo attention              | Scalabilità  | Generalità                       |
|------------------|------------------------------|--------------|----------------------------------|
| CNN              | $O(MKL)$ — lineare           | alta         | <b>bassa</b> (arch. per dominio) |
| Transformer      | $O(M^2)$ — <b>quadratico</b> | <b>bassa</b> | alta                             |
| <b>Perceiver</b> | $O(MN) + O(N^2)$ — lineare   | <b>alta</b>  | <b>alta</b>                      |

## Riduzione della complessità

$$\underbrace{O(M^2)}_{\text{Self-attention (Transformer)}} \longrightarrow \underbrace{O(MN)}_{\text{Cross-attention}} + \underbrace{O(N^2)}_{\text{Self-attention latente}}$$

Con  $N = 512$  e  $M = 50\,176$  il fattore di riduzione è  $\approx 100\times$ . Poiché  $N$  è *fisso*, il costo cresce solo *linearmente* in  $M$ .

## Idea chiave

Un solo modello per tutte le modalità, a costo **lineare** nell'input. Il prezzo da pagare: l'input va prima *compresso* in  $N \ll M$  latenti. Tutto il resto dell'architettura discende da questa scelta.

## 2 L'idea centrale: dalla self- alla cross-attention

### In breve

**Self-attention:**  $Q, K, V$  vengono tutti dallo stesso input  $\Rightarrow$  matrice di attenzione  $M \times M$ . **Cross-attention:**  $Q$  viene dai *latenti*,  $K, V$  dall'*input*  $\Rightarrow$  matrice  $N \times M$ . I latenti sono *parametri appresi*, con  $N \ll M$ : è questa asimmetria a rompere la quadraticità.

Nel Transformer standard la **self-attention** calcola Query, Key e Value *tutti dallo stesso* input  $X$ : ogni token guarda tutti gli altri, producendo una matrice di attenzione **quadrata**  $M \times M$ .

Il Perceiver la sostituisce con la **cross-attention**, dove le **Query provengono da un array diverso** — i latenti  $L$  — mentre Key e Value continuano a venire dall'input  $X$ . La matrice di attenzione diventa **rettangolare**  $N \times M$ , e il costo crolla da  $O(M^2)$  a  $O(MN)$ .

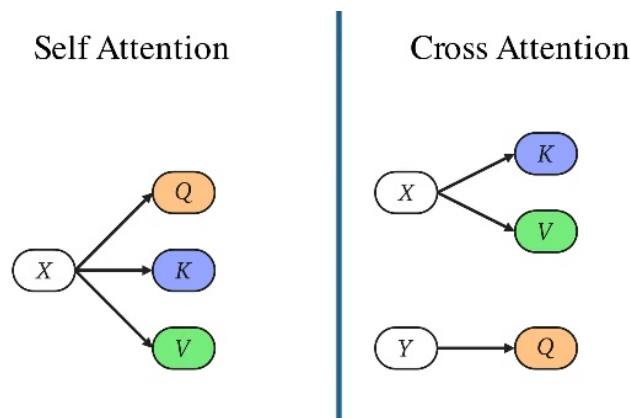


Figura 1: Self-Attention vs Cross-Attention. Nella self-attention  $Q, K, V$  vengono tutti dallo stesso input  $X$ . Nella cross-attention  $K, V$  vengono dall'input  $X$ , mentre  $Q$  viene da un altro array (i latenti  $L$ ).

| Aspetto            | Transformer (Self-Att)  | Perceiver (Cross-Att)       |
|--------------------|-------------------------|-----------------------------|
| Sorgente $Q$       | Input $X$ ( $M$ token)  | Latenti $L$ ( $N$ latenti)  |
| Sorgente $K, V$    | Input $X$ ( $M$ token)  | Input $X$ ( $M$ token)      |
| Matrice attenzione | $M \times M$ (quadrata) | $N \times M$ (rettangolare) |
| Complessità        | $O(M^2)$                | $O(MN)$ , con $N \ll M$     |

### Intuizione di $Q, K, V$

**Query** = una *domanda* (“cosa sto cercando?”). **Key** = un'*etichetta* (“io rappresento questo”).  
**Value** = il *contenuto* informativo vero e proprio. L'attenzione confronta ogni domanda con tutte le etichette e restituisce una media pesata dei contenuti più pertinenti.

I **latenti**  $L$  sono *parametri appresi* del modello (inizializzati come learned position encodings, paper App. C): **non derivano dall'input** e non hanno un legame spaziale con pixel o token specifici. Sono una *base compatta* — delle “unità di memoria” — che il modello usa per riassumere qualsiasi input.

### Idea chiave

La cross-attention è un piccolo gruppo di  $N$  latenti che “**interroga**” i  $M$  elementi dell'input:  $Q$  dai latenti (*cosa cerco?*),  $K/V$  dall'input (*cosa contengo?*). L'asimmetria  $N \ll M$  è ciò

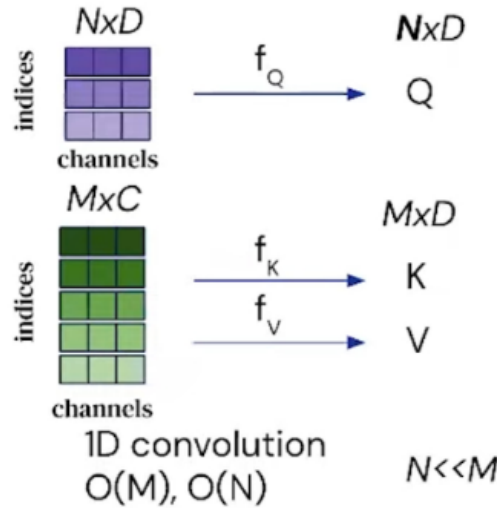


Figura 2: Cross-attention, primo step: il *latent array* ( $N \times D$ ) genera le Query  $Q$ , mentre il *byte array* ( $M \times C$ ) genera Key  $K$  e Value  $V$ . Sono le *sorgenti diverse* di  $Q$  e di  $K, V$  a definire la cross-attention.

che rompe la quadraticità.

### 3 Architettura in tre stadi

#### In breve

Tre stadi: **(1)** un encoder a cross-attention *comprime* l'input nei latenti; **(2)** un latent transformer *raffina* i latenti con self-attention; **(3)** una testa di pooling + classificatore produce i logit. Il blocco (1)+(2) si ripete  $T$  volte.

L'idea di fondo è **separare la dimensione dei dati in ingresso dalla capacità della rete**. Un input arbitrariamente grande viene prima proiettato in un latent array di dimensione fissa, e solo su quello la rete fa la sua elaborazione profonda. Ne discendono le due proprietà che *definiscono* il Perceiver:

1. lo stesso modello gestisce modalità molto diverse **senza modifiche**;
2. la **profondità della rete è disaccoppiata** dalla lunghezza dell'input.

L'elaborazione si snoda in **tre stadi**:

1. **Encoder a cross-attention** — legge l'input  $X \in \mathbb{R}^{M \times C_{\text{tot}}}$  e lo *comprime* nel latent array  $L \in \mathbb{R}^{N \times D}$ .
2. **Latent transformer** — raffina  $L$  con self-attention profonda, interamente nello spazio compresso.
3. **Testa di pooling + classificazione** — aggrega i latenti e produce i logit.

Il blocco *cross-attention* + *latent transformer* viene poi ripetuto  $T$  volte (con weight sharing, §4.6).

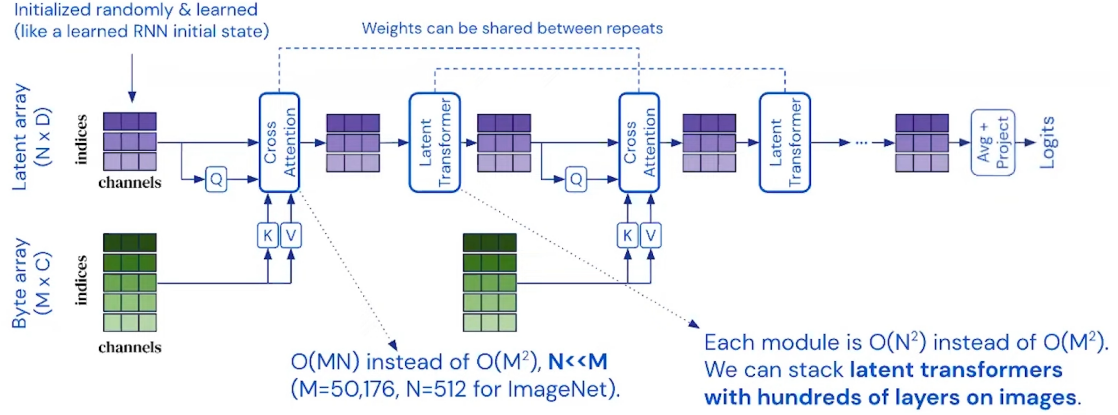


Figura 3: Architettura del Perceiver. Il *Byte Array* ( $M \times C$ ) entra da sinistra, viene letto dal *Latent Array* ( $N \times D$ ) tramite Cross-Attention, poi i latenti sono raffinati dal *Latent Transformer*, infine mediati (Average + Project) per produrre i *Logits*. I pesi sono condivisi tra le  $T$  iterazioni (linea tratteggiata in alto).

Tabella 1: Iperparametri del Perceiver nella configurazione ImageNet (Jaegle et al., 2021, §4.1 + App. C). Questi valori ricorrono in tutto il forward pass.

| Simbolo          | Valore | Significato                                                    |
|------------------|--------|----------------------------------------------------------------|
| $M$              | 50 176 | elementi di input ( $224 \times 224$ pixel)                    |
| $C$              | 3      | canali grezzi (RGB)                                            |
| $C_{\text{tot}}$ | 261    | canali per elemento dopo le Fourier features                   |
| $N$              | 512    | numero di latenti ( $N \ll M$ )                                |
| $D$              | 1024   | dimensione di ogni latente                                     |
| $d_{QKV}$        | 261    | dim. interna cross-attention = $\min(D, C_{\text{tot}})$       |
| $T$              | 8      | iterazioni (cross-att + latent transformer)                    |
| $\ell$           | 6      | self-attention per blocco del latent transformer               |
| $H$              | 8      | teste della self-attention latente ( $d_{\text{head}} = 128$ ) |
| $K$              | 64     | bande di frequenza Fourier per asse                            |

Ogni stadio è una sottosezione qui sotto. La **striscia in cima alla pagina** segue il tuo punto nella pipeline.

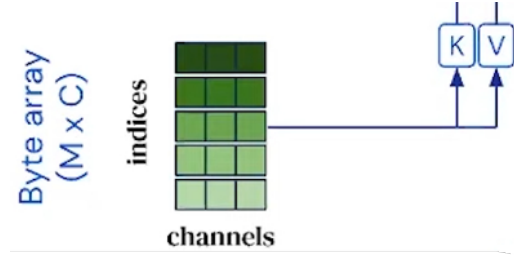
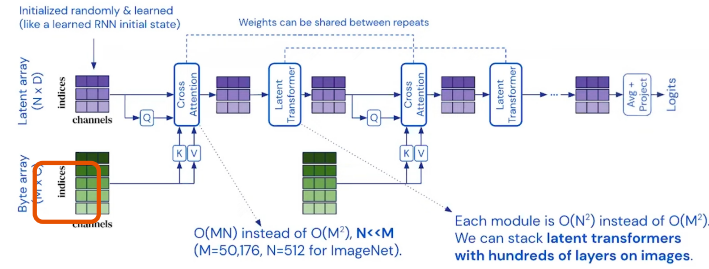


Figura 4: Il byte array ( $M \times C$ ): ogni riga è un elemento dell'input (un pixel) con i suoi  $C$  canali. Da qui si generano le Key e i Value della cross-attention.

## 4.2 Positional encoding: le Fourier features



Sei qui: *Byte array + Fourier PE*

**Problema:** l'attention è *cieca alla posizione*. Un pixel  $[R, G, B]$  dice solo il colore, non *dove* si trova. Bisogna iniettare la posizione esplicitamente.

**Soluzione:** si arricchisce ogni elemento con le **Fourier features** — la posizione  $x$  viene mappata in uno spazio ad alta dimensione applicando seno e coseno a  $K$  frequenze diverse.

### Fourier positional encoding

Per ogni asse spaziale, dato il valore di coordinata  $x_d \in [-1, 1]$ :

$$\text{PE}(x_d) = \left[ \underbrace{\sin(f_1 \pi x_d), \cos(f_1 \pi x_d), \dots, \sin(f_K \pi x_d), \cos(f_K \pi x_d)}_{2K \text{ valori sinusoidali}}, \underbrace{x_d}_{\text{coord. grezza}} \right]$$

Tre scelte di design da ricordare:

- **Frequenze *linearly spaced*** tra 1 e  $\mu/2$  (la frequenza di Nyquist), *non* esponenziali come in NeRF ( $f_k = 2^k$ ). Con  $K = 64$  bande, NeRF darebbe  $2^{64} \approx 10^{19}$ : numeri ingestibili, e il paper riporta instabilità di training già oltre  $k \approx 15$  (App. D).
- **Si concatena anche la coordinata grezza  $x_d$** : le sinusoidi sono *periodiche* (quindi ambigue — frequenze alte mappano posizioni diverse nello stesso valore), mentre la coordinata grezza dà l'*identità* posizionale esatta.
- **Si *concatena* al colore, non si *somma***: la concatenazione tiene colore e posizione separati. (I Transformer per il linguaggio invece *sommano* il positional encoding.)

### Esempio numerico — un pixel a posizione (0.3, 0.7)

Pixel rosso, posizione normalizzata  $(x, y) = (0.3, 0.7)$ .

- Colore RGB normalizzato:  $[1.0, 0, 0]$ .



- Fourier per l'asse  $x = 0.3$ , con 2 frequenze illustrative  $f_1=1, f_2=2$  (il paper ne usa 64):

$$\text{PE}(0.3) = \underbrace{[0.3, \sin(0.3\pi), \cos(0.3\pi), \sin(0.6\pi), \cos(0.6\pi)]}_x = [0.3, 0.809, 0.588, 0.951, -0.309]$$

- Analogamente per l'asse  $y = 0.7$ .
- Vettore finale del pixel:  $\underbrace{[1.0, 0, 0]}_{\text{RGB}}, \underbrace{[0.3, \dots]}_{129 \text{ (asse } x)}, \underbrace{[0.7, \dots]}_{129 \text{ (asse } y)} \in \mathbb{R}^{261}$ .

#### Da dove viene $C_{\text{tot}} = 261$

Per ogni asse:  $2K + 1 = 2 \cdot 64 + 1 = 129$  valori (128 sinusoidi + 1 coordinata grezza).

Con  $d = 2$  assi (x e y):  $2 \times 129 = 258$  valori posizionali.

Sommati ai 3 canali RGB:

$$C_{\text{tot}} = \underbrace{3}_{\text{RGB}} + \underbrace{d \times (2K + 1)}_{2 \times 129 = 258 \text{ Fourier}} = \mathbf{261}$$

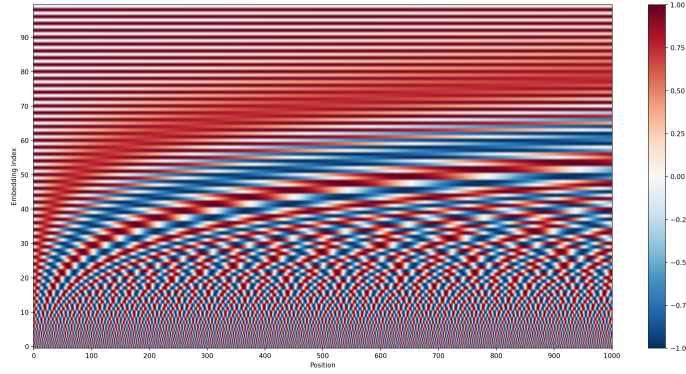


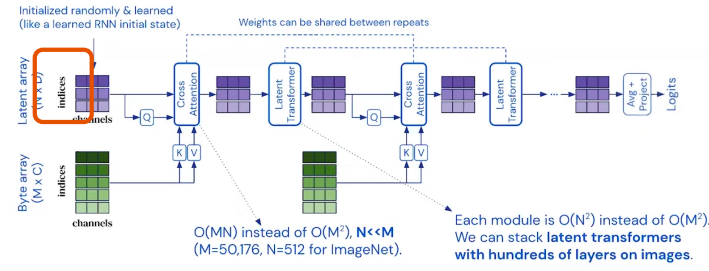
Figura 5: Le Fourier features mappano una posizione scalare in un vettore di sinusoidi a frequenze multiple: le basse frequenze catturano la posizione globale, le alte i dettagli fini.

| Oggetto                                           | Forma simbolica           | ImageNet                               | Contenuto di una riga      |
|---------------------------------------------------|---------------------------|----------------------------------------|----------------------------|
| Byte array grezzo                                 | $M \times C$              | $50\,176 \times 3$                     | RGB                        |
| <b>Byte array <math>X</math> (con Fourier PE)</b> | $M \times C_{\text{tot}}$ | <b><math>50\,176 \times 261</math></b> | <b>3 RGB + 258 Fourier</b> |

#### Idea chiave

Le Fourier features danno al modello la **posizione**. È questa scelta — non l'architettura in sé — a rendere il Perceiver **invariante alle permutazioni** dei pixel: la posizione “viaggia” dentro l'input, non nell'ordine degli indici (vedi §7).

### 4.3 Il latent array



Sei qui: *Latent array*

Il **latent array**  $L \in \mathbb{R}^{N \times D}$  è una matrice di  $N$  vettori latenti, **interamente appresa** (sono parametri del modello). Non deriva dall'input: è uguale per tutti gli esempi del batch e cambia solo durante il training.

#### Inizializzazione dei latenti

Per ImageNet  $L \in \mathbb{R}^{512 \times 1024}$ . Ogni elemento è campionato da una gaussiana stretta:

$$L_{ij} \sim \mathcal{N}(0, 0.02^2), \quad \text{troncata a } [-2, 2].$$

I valori sono quasi tutti minuscoli ( $\sim \pm 0.02$ ). **Perché così piccoli?**

- **Stabilità:** latenti grandi  $\rightarrow$  prodotti  $QK^\top$  enormi  $\rightarrow$  softmax satura  $\rightarrow$  gradienti instabili.
- **Neutralità:** media zero  $\Rightarrow$  nessun latente parte avvantaggiato.
- **Specializzazione graduale:** partendo "neutri", i latenti si riempiono progressivamente di informazione e col tempo ciascuno si specializza (bordi, colori, pattern globali...).

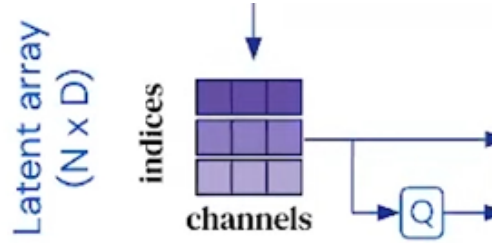


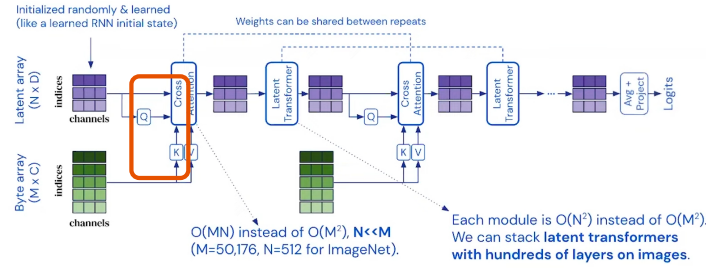
Figura 6: Il latent array ( $N \times D$ ): matrice appresa che genera le Query ( $Q$ ) per la cross-attention.

A questo punto abbiamo i **due array** che la cross-attention metterà in comunicazione:

| Oggetto                            | Forma                          | ImageNet                            | Natura                                           |
|------------------------------------|--------------------------------|-------------------------------------|--------------------------------------------------|
| Byte array $X$                     | $M \times C_{\text{tot}}$      | $50\,176 \times 261$                | input (3 RGB + 258 Fourier)                      |
| <b>Latent array <math>L</math></b> | <b><math>N \times D</math></b> | <b><math>512 \times 1024</math></b> | parametri appresi, init $\mathcal{N}(0, 0.02^2)$ |

Nota: i due array hanno dimensione-feature *diversa* (261 vs 1024); sarà la cross-attention a farli incontrare. E il numero di righe è molto diverso (50 176 vs 512): è questa asimmetria a dare la complessità  $O(MN)$ .

## 4.4 Il blocco di cross-attention



Sei qui: *Cross-Attention*

Qui gli  $N$  latenti “leggono” informazione dagli  $M$  elementi dell’input. Vediamo i passaggi con le forme esatte.

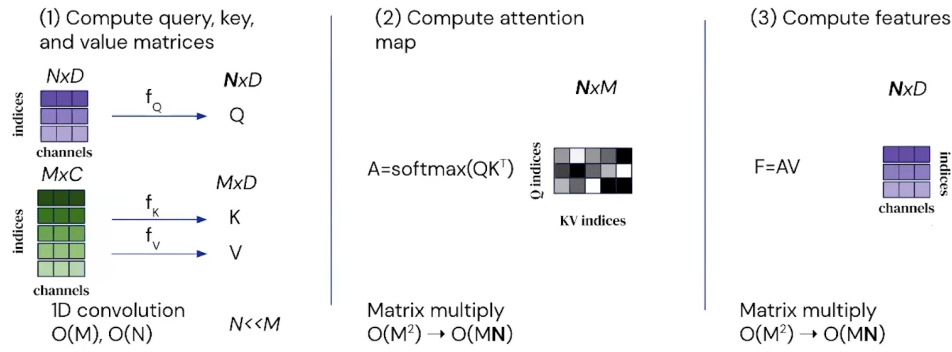


Figura 7: Cross-attention, vista d’insieme dei 3 step: (1) si calcolano le matrici  $Q, K, V$ ; (2) si calcola la mappa di attenzione  $N \times M$ ; (3) si aggregano le feature.

**Passo 1 — LayerNorm pre-attention.** Entrambi gli array attraversano una **LayerNorm** prima delle proiezioni (pattern *pre-norm*, paper App. C). La LN normalizza *ogni riga* a media 0 e varianza 1, poi applica due parametri appresi  $\gamma, \beta$ . Non cambia le forme:  $\tilde{X} \in \mathbb{R}^{50176 \times 261}$ ,  $\tilde{L} \in \mathbb{R}^{512 \times 1024}$ .

*Perché pre-norm?* Il residuo resta un percorso “pulito” su cui il gradiente fluisce senza attraversare la LN  $\Rightarrow$  training stabile su reti profonde (lo stesso schema di GPT-2).

**Passo 2 — dimensione interna  $d_{QKV}$ .**  $Q, K, V$  devono avere la stessa dimensione-feature, altrimenti  $QK^\top$  non è definito. Si sceglie:

$$d_{QKV} = \min(D, C_{\text{tot}}) = \min(1024, 261) = \mathbf{261}.$$

*Perché il minimo?* L’input ha già solo 261 canali: proiettare  $K, V$  in una dimensione più grande non aggiungerebbe informazione, sarebbe solo spreco di parametri e FLOPs.

**Passo 3 — proiezioni  $Q, K, V$ .** Tre matrici di pesi apprese (*senza bias*, paper App. C):

### Proiezioni lineari

$$\begin{aligned} Q &= \tilde{L} W_Q, & W_Q &\in \mathbb{R}^{D \times d_{QKV}} = \mathbb{R}^{1024 \times 261}, & Q &\in \mathbb{R}^{512 \times 261} & (\text{dai latenti}) \\ K &= \tilde{X} W_K, & W_K &\in \mathbb{R}^{C_{\text{tot}} \times d_{QKV}} = \mathbb{R}^{261 \times 261}, & K &\in \mathbb{R}^{50176 \times 261} & (\text{dall'input}) \\ V &= \tilde{X} W_V, & W_V &\in \mathbb{R}^{C_{\text{tot}} \times d_{QKV}} = \mathbb{R}^{261 \times 261}, & V &\in \mathbb{R}^{50176 \times 261} & (\text{dall'input}) \end{aligned}$$

Le matrici  $W_Q, W_K, W_V$  sono *apprese*: imparano a produrre query, key e value “ottimali” per il matching (es. riducendo il peso di feature rumorose). I pesi sono inizializzati con *Xavier Uniform*

(Glorot & Bengio, 2010), che mantiene costante la varianza delle attivazioni attraverso i layer. **L'asimmetria chiave:**  $Q$  ha *poche* righe ( $N = 512$ ),  $K, V$  ne hanno *tante* ( $M = 50176$ ).

**Passo 4 — scaled dot-product attention.** Le quattro operazioni che combinano  $Q, K, V$ :

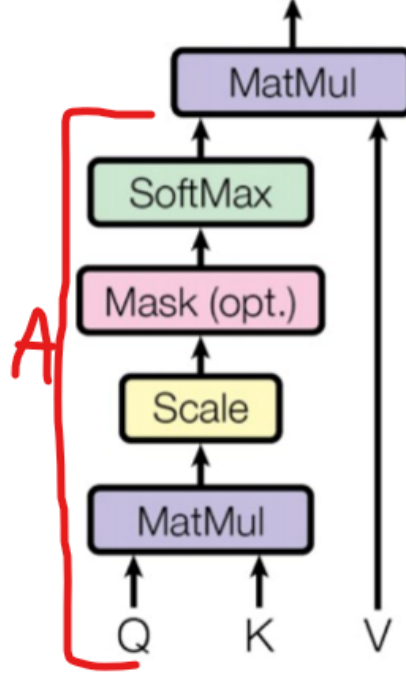


Figura 8: Pipeline della scaled dot-product attention: prodotto  $QK^\top$ , scaling per  $\sqrt{d_{QKV}}$ , softmax per riga, prodotto finale con  $V$ . (Il passo “Mask” non è usato: il Perceiver è non-causale.)

#### Scaled dot-product

- |                      |                                                                                              |
|----------------------|----------------------------------------------------------------------------------------------|
| (a) score            | $S = QK^\top \in \mathbb{R}^{512 \times 50176}$                                              |
| (b) scaling          | $S' = S / \sqrt{d_{QKV}} = S / \sqrt{261} \approx S / 16$                                    |
| (c) softmax per riga | $A_{ij} = e^{S'_{ij}} / \sum_{k=1}^M e^{S'_{ik}}, \quad A \in \mathbb{R}^{512 \times 50176}$ |
| (d) aggregazione     | $F = AV \in \mathbb{R}^{512 \times 261}$                                                     |

- **(a)** ogni entry  $(i, j)$  misura “quanto il latente  $i$  trova interessante il pixel  $j$ ”.
- **(b)** senza scaling gli score avrebbero deviazione standard  $\sqrt{d_{QKV}} \approx 16$ : la softmax saturerebbe (tutta la massa su un pixel) e i gradienti si annullerebbero.
- **(c)** la softmax rende i pesi non-negativi e con somma 1 su ogni riga  $\Rightarrow$  ogni latente ha un “budget di attenzione” da distribuire sui 50176 pixel.
- **(d)** ogni latente diventa una *media pesata* dei Value dell’input.

**Passo 5 — output projection e residual.**  $F$  vive in dimensione 261; va riportato a  $D = 1024$  per poterlo sommare ai latenti:

$$Z' = F W_o + b_o, \quad W_o \in \mathbb{R}^{261 \times 1024} \Rightarrow Z' \in \mathbb{R}^{512 \times 1024}; \quad L' = L + Z' \in \mathbb{R}^{512 \times 1024}.$$

**Passo 6 — MLP (feed-forward).** Pre-norm, espansione a  $4D$ , attivazione GELU, compressione, residual:

$$\tilde{L}' = \text{LN}(L'); \quad H = \tilde{L}' W_1 + b_1 \in \mathbb{R}^{512 \times 4096}; \quad \phi = \text{GELU}(H) W_2 + b_2 \in \mathbb{R}^{512 \times 1024}; \quad \hat{L} = L' + \phi.$$

(2) Compute attention map

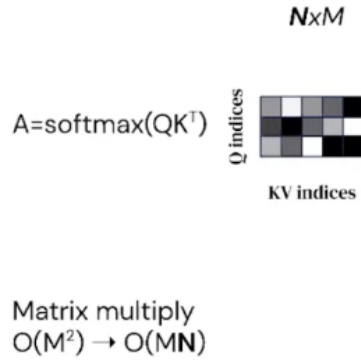


Figura 9: La mappa di attenzione  $A = \text{softmax}(QK^\top / \sqrt{d_{QKV}})$  è **rettangolare**  $N \times M = 512 \times 50176$  (righe = latenti, colonne = pixel) — non quadrata  $M \times M$ . È qui che la complessità scende da  $O(M^2)$  a  $O(MN)$ .

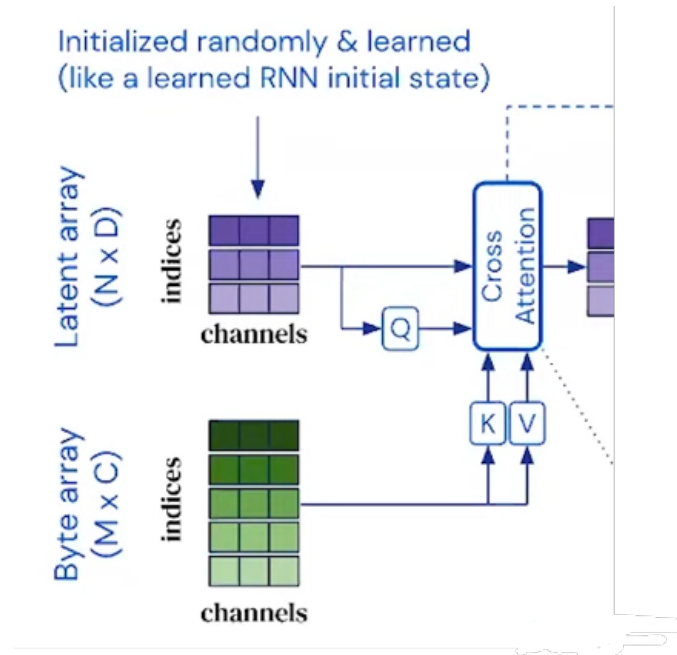


Figura 10: Schema complessivo della cross-attention: il latent array ( $512 \times 1024$ ) proiettato a  $Q$  ( $512 \times 261$ ) interroga il byte array ( $50176 \times 261$ ) proiettato a  $K, V$ . Il prodotto  $QK^\top$  dà una matrice di attenzione rettangolare  $512 \times 50176$ .

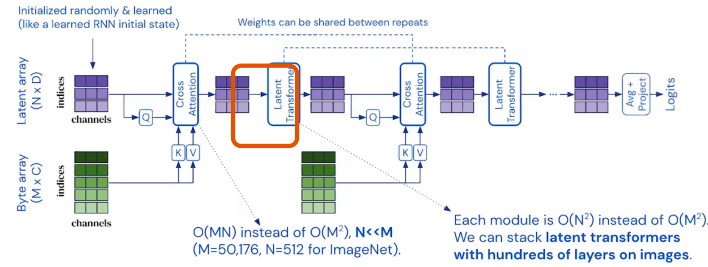
**Idea chiave**

Output del blocco:  $\hat{L} \in \mathbb{R}^{512 \times 1024}$ , **stessa forma di  $L$** . Il blocco non cambia la dimensione del latent array, solo il suo *contenuto* — ora arricchito dell'informazione letta dall'input. È questa invarianza di forma che permette di ripetere il blocco  $T$  volte.

Tabella 2: Riepilogo completo del blocco Cross-Attention + MLP (ImageNet). Input:  $L \in \mathbb{R}^{512 \times 1024}$ ,  $X \in \mathbb{R}^{50 \times 176 \times 261}$ .

| #                                      | Operazione         | Formula                                                                            | Dimensione                              |
|----------------------------------------|--------------------|------------------------------------------------------------------------------------|-----------------------------------------|
| <b>Cross-Attention</b>                 |                    |                                                                                    |                                         |
| 1                                      | LayerNorm pre-att. | $\tilde{L} = \text{LN}(L)$ , $\tilde{X} = \text{LN}(X)$                            | invariate                               |
| 2                                      | Proiezione $Q$     | $Q = \tilde{L} W_Q$ , $W_Q \in \mathbb{R}^{1024 \times 261}$                       | $\mathbb{R}^{512 \times 261}$           |
|                                        | Proiezione $K$     | $K = \tilde{X} W_K$ , $W_K \in \mathbb{R}^{261 \times 261}$                        | $\mathbb{R}^{50 \times 176 \times 261}$ |
|                                        | Proiezione $V$     | $V = \tilde{X} W_V$ , $W_V \in \mathbb{R}^{261 \times 261}$                        | $\mathbb{R}^{50 \times 176 \times 261}$ |
| 3                                      | Scaled scores      | $S = QK^\top / \sqrt{261}$                                                         | $\mathbb{R}^{512 \times 50 \times 176}$ |
| 4                                      | Softmax per riga   | $A_{ij} = e^{S_{ij}} / \sum_k e^{S_{ik}}$                                          | $\mathbb{R}^{512 \times 50 \times 176}$ |
| 5                                      | Aggregazione       | $F = AV$                                                                           | $\mathbb{R}^{512 \times 261}$           |
| 6                                      | Output projection  | $Z' = F W_o + b_o$ , $W_o \in \mathbb{R}^{261 \times 1024}$                        | $\mathbb{R}^{512 \times 1024}$          |
| 7                                      | Residual           | $L' = L + Z'$                                                                      | $\mathbb{R}^{512 \times 1024}$          |
| <b>MLP (pre-norm, GELU, hidden 4D)</b> |                    |                                                                                    |                                         |
| 8                                      | LayerNorm pre-MLP  | $\tilde{L}' = \text{LN}(L')$                                                       | $\mathbb{R}^{512 \times 1024}$          |
| 9                                      | Espansione + GELU  | $H' = \text{GELU}(\tilde{L}' W_1 + b_1)$ , $W_1 \in \mathbb{R}^{1024 \times 4096}$ | $\mathbb{R}^{512 \times 4096}$          |
|                                        | Compressione       | $\phi = H' W_2 + b_2$ , $W_2 \in \mathbb{R}^{4096 \times 1024}$                    | $\mathbb{R}^{512 \times 1024}$          |
| 10                                     | Residual post-MLP  | $\hat{L} = L' + \phi$                                                              | $\mathbb{R}^{512 \times 1024}$          |

## 4.5 Il latent transformer



Sei qui: *Latent Transformer*

Una volta che i latenti contengono l'informazione, vengono **raffinati** da una pila di  $\ell$  blocchi di **self-attention** che operano *solo sui latenti* ( $N \times D$ ), senza più toccare l'input. Qui i latenti “comunicano tra loro”: è il passo che *integra* l'informazione raccolta dalla cross-attention.

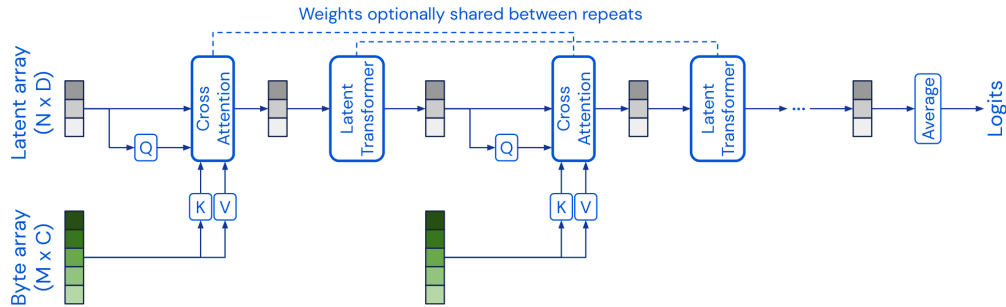


Figura 11: Il latent transformer nella pipeline del Perceiver (Jaegle et al., 2021, Fig. 1): si alterna alla cross-attention e opera *solo* sui latenti tramite self-attention profonda.

Struttura di un blocco (architettura **GPT-2**, Radford et al., 2019): LayerNorm → multi-head self-attention → residual → LayerNorm → MLP (GELU, hidden  $4D$ ) → residual. Non causale — nessuna maschera.

**Self-attention multi-head** ( $H = 8$  teste,  $d_{\text{head}} = D/H = 128$ ). Qui  $Q, K, V$  vengono *tutti* dai latenti:

$$Q = \tilde{L} W_q, \quad K = \tilde{L} W_k, \quad V = \tilde{L} W_v \quad (W \in \mathbb{R}^{1024 \times 128} \text{ per testa}); \quad A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_{\text{head}}}}\right) \in \mathbb{R}^{512 \times 512}.$$

I risultati delle 8 teste vengono concatenati ( $512 \times 1024$ ) e proiettati da  $W_O$ .

### Multi-head: perché più teste?

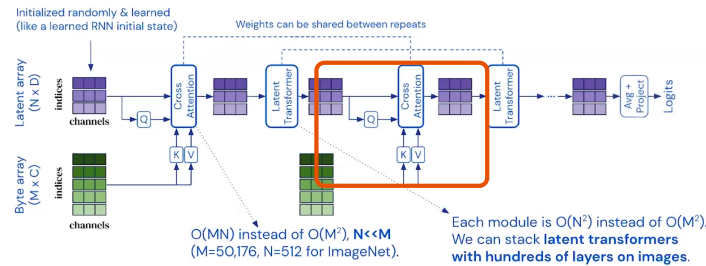
Invece di una sola attention in dimensione  $D$ , se ne calcolano  $H$  in parallelo, ciascuna in dimensione  $d_{\text{head}} = D/H$ . Ogni testa può specializzarsi su un tipo di relazione diverso (es. una guarda pattern locali, un'altra globali). I risultati si concatenano: il costo totale resta lo stesso di una singola attention in dimensione  $D$ .

| Aspetto            | Cross-attention                   | Latent transformer                      |
|--------------------|-----------------------------------|-----------------------------------------|
| Sorgente $Q$       | latenti                           | latenti                                 |
| Sorgente $K, V$    | <b>input</b> $X$                  | <b>latenti</b>                          |
| Tipo               | cross-attention                   | self-attention                          |
| Dim. interna       | $d_{QKV} = 261$                   | $D = 1024$ (8 teste da 128)             |
| Matrice attenzione | $512 \times 50176$ (rettangolare) | $512 \times 512$ (quadrata)             |
| Costo              | $O(MN)$                           | $O(N^2)$ — piccolo grazie al bottleneck |

### Idea chiave

La **profondità** del modello vive qui, nello spazio latente compresso. La matrice di attenzione è  $N \times N = 512 \times 512$ : impilare molti layer costa  $O(N^2)$ , *non*  $O(M^2)$ . Ecco perché la profondità della rete è **disaccoppiata** dalla dimensione dell'input.

## 4.6 Weight sharing e iterazioni



Sei qui: Iterazioni  $\times T$  (weight sharing)

È una delle idee più importanti del Perceiver. Il blocco *cross-attention* + *latent transformer* viene riapplicato  $T = 8$  volte agli stessi latenti, **condividendo i pesi** tra le iterazioni — con un'eccezione: il *primo* blocco ha pesi propri.

### Perché il primo blocco è escluso dallo sharing

La prima cross-attention riceve i latenti nella loro inizializzazione “neutra” ( $\mathcal{N}(0, 0.02^2)$ ); dalla seconda in poi legge invece latenti già “popolati” di informazione. Due regimi così diversi non possono condividere gli stessi pesi senza causare **instabilità di training**. Quindi  $CA_1, LT_1$  hanno pesi unici;  $CA_2, \dots, CA_T$  e  $LT_2, \dots, LT_T$  condividono ciascuno un set di pesi.

Questo rende il Perceiver **formalmente un RNN**:

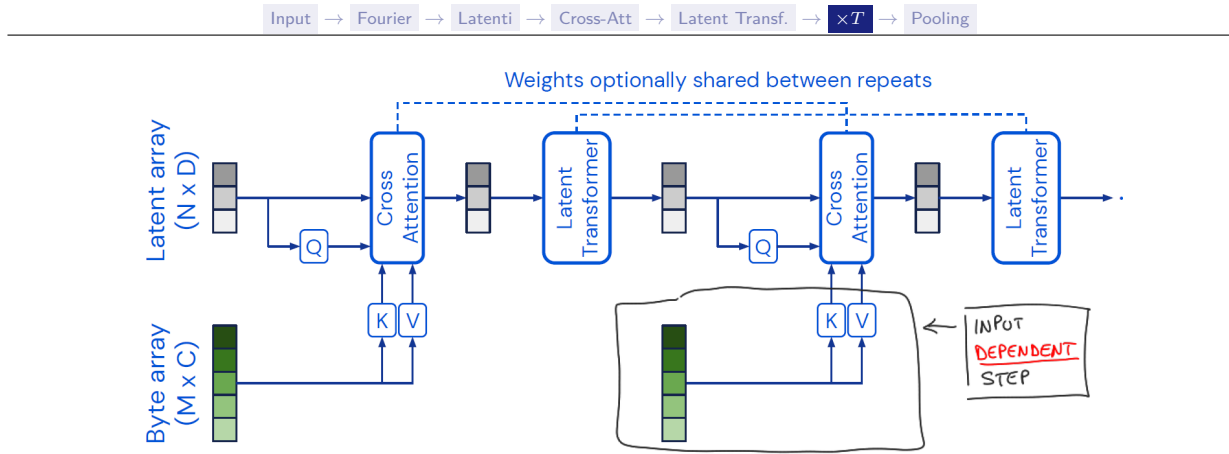


Figura 12: Il Perceiver completo con weight sharing (Jaegle et al., 2021, Fig. 1): i blocchi Cross-Attention e Latent Transformer si ripetono  $T$  volte condividendo i pesi (linee tratteggiate).

| Componente RNN             | Nel Perceiver                                                   |
|----------------------------|-----------------------------------------------------------------|
| stato ricorrente           | il latent array $L^{(t)}$ (dimensione fissa $512 \times 1024$ ) |
| input (costante nel tempo) | il byte array $X$                                               |
| funzione di transizione    | il blocco condiviso (cross-att + latent transformer)            |
| stato iniziale             | $L^{(0)}$ (parametro appreso)                                   |

A differenza di un RNN classico, però, a *ogni* passo i latenti accedono all'intero input **simultaneamente**, non a un elemento alla volta.

#### Esempio numerico — gradienti di un peso condiviso

Sia  $w = 2.0$  un peso condiviso tra le iterazioni  $t = 2$  e  $t = 3$ . Nel backward ogni istanza produce un contributo:

$$\frac{\partial \mathcal{L}}{\partial w^{(2)}} = -0.3, \quad \frac{\partial \mathcal{L}}{\partial w^{(3)}} = +0.1.$$

I contributi si **sommano** (come in un RNN srotolato — BPTT):

$$\frac{\partial \mathcal{L}}{\partial w^{\text{sh}}} = -0.3 + 0.1 = -0.2.$$

Con learning rate  $\eta = 0.01$ :  $w \leftarrow w - \eta \cdot (-0.2) = 2.0 + 0.002 = \mathbf{2.002}$ .

*Attenzione:* l'accumulazione va fatta *prima* del passo dell'ottimizzatore. In PyTorch è automatica (più nodi del grafo che puntano allo stesso `Parameter`).

#### Effetto del weight sharing (Tab. 7 del paper)

**Senza** weight sharing:  $\sim 326\text{M}$  parametri    **72.9%** accuracy (forte overfitting)

**Con** weight sharing:  $\sim 45\text{M}$  parametri    **78.0%** accuracy

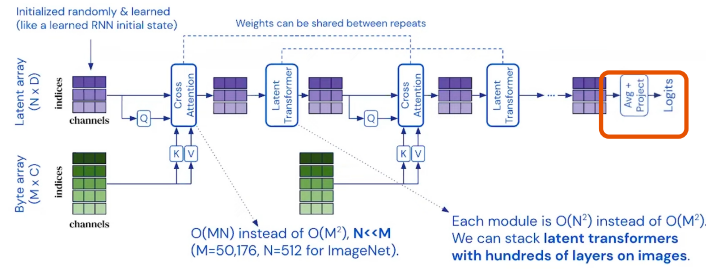
$\Rightarrow \sim 7\times$  **meno parametri** e *contemporaneamente* **+5.1%** di accuracy.

#### Idea chiave

Il weight sharing trasforma il Perceiver in un RNN che **raffina progressivamente** i latenti, rileggendo l'input  $T$  volte con lo stesso “modulo di raffinamento”. Agisce da **regolarizzatore**: meno parametri, niente overfitting, più accuratezza.



## 4.7 Output: pooling e classificazione



Sei qui: *Avg + Project* → *Logits*

Dopo le  $T$  iterazioni, dai latenti finali  $L^{(T)} \in \mathbb{R}^{512 \times 1024}$  si arriva alla predizione in tre passi.

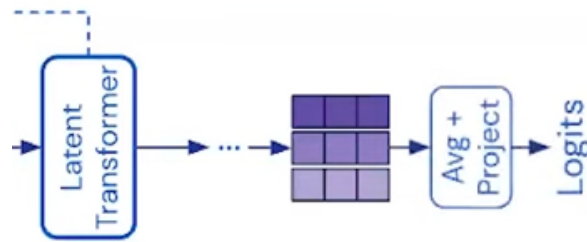


Figura 13: Testa di output: i latenti finali vengono mediati (Average) e proiettati (Project) nei logit per la classificazione.

### Dai latenti alla classe predetta (ImageNet, 1000 classi)

1. **Global average pooling** — media degli  $N$  latenti finali:

$$h = \frac{1}{N} \sum_{i=1}^N L_i^{(T)} \in \mathbb{R}^D = \mathbb{R}^{1024}.$$

2. **Classificatore lineare** — proietta nello spazio delle classi:

$$z = h W_c + b_c, \quad W_c \in \mathbb{R}^{1024 \times 1000}, \quad b_c \in \mathbb{R}^{1000} \Rightarrow z \in \mathbb{R}^{1000} \text{ (logit)}.$$

3. **Softmax + argmax** — logit → probabilità → classe:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{1000} e^{z_j}}, \quad \hat{y} = \arg \max_i p_i.$$

A differenza di BERT/GPT, il Perceiver **non usa un token [CLS]**: il pooling medio è la soluzione più diretta e *non introduce nuovi parametri*. Il pooling è fatto *prima* della proiezione lineare (un fix del paper, App. F).

### Idea chiave

Il pooling medio comprime gli  $N$  latenti in *un solo* vettore  $h \in \mathbb{R}^D$ , di dimensione fissa — qualunque sia  $N$ . Questo permette di collegare un classificatore lineare standard, identico per qualsiasi modalità di input.

## E il backward pass?

Il modello è allenato end-to-end con la *backpropagation* standard applicata a ogni blocco. Con il weight sharing, i gradienti dei blocchi condivisi si **sommano** prima del passo dell’ottimizzatore (come in un RNN srotolato — BPTT, vedi esempio in §4.6). La derivazione completa dei gradienti è meccanica e non introduce concetti teorici nuovi: per la *teoria* del Perceiver basta la pipeline forward qui sopra.

## 5 I componenti ricorrenti

### In breve

Il Perceiver riusa quattro “mattoni” standard del deep learning: **LayerNorm** (normalizza), **GELU** (la non-linearità delle MLP), la **residual connection** (la somma  $x + f(x)$ ) e la **softmax** (da punteggi a probabilità). Compaiono in *ogni* blocco del forward pass.

Il Perceiver non introduce nuovi “pezzi” a basso livello: la sua novità sta nell’*architettura* (cross-attention, latent bottleneck, weight sharing), non nei singoli componenti. Questi ultimi sono quattro elementi standard che conviene conoscere bene, perché ricorrono in tutta la pipeline.

### 5.1 LayerNorm — normalizzazione

La **Layer Normalization** stabilizza la distribuzione dei valori che entrano in attention e MLP. Agisce *riga per riga*: per ogni elemento (un latente, o un pixel)  $x = (x_1, \dots, x_F)$  calcola media  $\mu$  e deviazione standard  $\sigma$  delle sue  $F$  feature, e applica:

$$\hat{x}_j = \gamma_j \cdot \frac{x_j - \mu}{\sigma + \epsilon} + \beta_j.$$

Prima si normalizza a media 0 e varianza 1 (*per costruzione matematica*); poi due parametri *appresi* per-feature,  $\gamma$  (scala) e  $\beta$  (shift), lasciano alla rete la libertà di ri-tarare la distribuzione. Il termine  $\epsilon \approx 10^{-5}$  evita la divisione per zero.

**Pre-norm.** Il Perceiver applica la LayerNorm *prima* di ogni sotto-blocco, non dopo:  $y = x + f(\text{LN}(x))$ . Così il ramo residuo resta un percorso “pulito” su cui il gradiente scorre senza attraversare la LN  $\Rightarrow$  training stabile anche con molti layer. Una LN precede *ogni* cross-attention, *ogni* self-attention e *ogni* MLP.

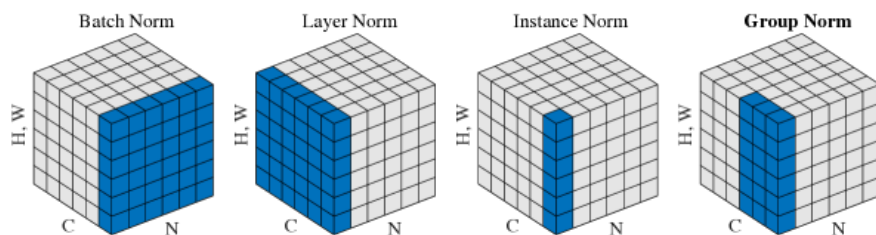


Figura 14: Metodi di normalizzazione a confronto. Il Perceiver usa la **LayerNorm**, che normalizza lungo le feature di *ogni singolo elemento* — indipendentemente dagli altri esempi del batch.

## 5.2 GELU — la funzione di attivazione

Dentro ogni **MLP** serve una *non-linearità*: senza, due layer lineari in sequenza collasserebbero in uno solo e la rete non potrebbe imparare funzioni complesse. Il Perceiver usa la **GELU** (Gaussian Error Linear Unit):

$$\text{GELU}(x) = x \cdot \Phi(x),$$

dove  $\Phi$  è la funzione di ripartizione della gaussiana standard: in pratica ogni valore viene pesato per la “probabilità di essere grande”.

**GELU vs ReLU.** La ReLU azzera di colpo tutti i valori negativi (un “gomito” netto in 0); la GELU è *liscia*, lascia passare una piccola parte dei valori leggermente negativi ed è differenziabile ovunque. La transizione morbida rende l’ottimizzazione più stabile. Il Perceiver adotta la GELU seguendo l’architettura GPT-2/BERT.

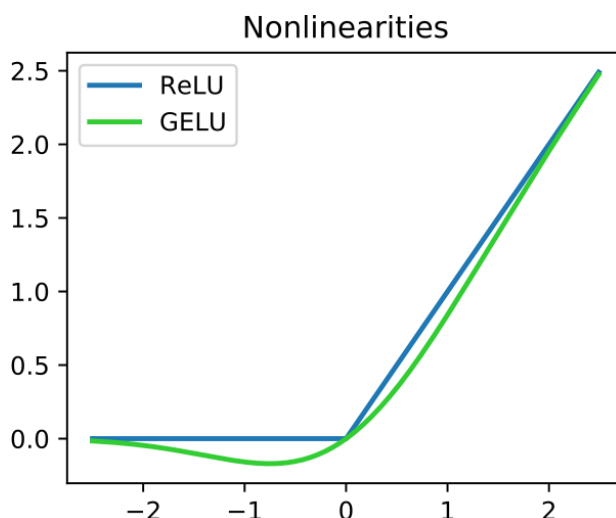


Figura 15: ReLU vs GELU. La ReLU ha un gomito netto in 0; la GELU è liscia e lascia passare leggermente i valori negativi vicini a zero.

## 5.3 Residual connection — la somma che salva il gradiente

Una **residual connection** (o *skip connection*) trasforma un blocco da  $\text{output} = f(x)$  a:

$$\text{output} = x + f(x),$$

cioè **somma l’input** al risultato del blocco. Sembra un dettaglio, ma è ciò che rende addestrabili le reti *profonde*: nel backward il gradiente trova un **percorso diretto** attraverso il “+” (la derivata della somma rispetto a  $x$  contiene un 1), quindi non si attenua strato dopo strato — niente *vanishing gradient*. È l’idea che ha reso possibili le ResNet.

Nel Perceiver *ogni* sotto-blocco è avvolto in una residual:  $L' = L + \text{Attention}(\text{LN}(L))$  e poi  $\hat{L} = L' + \text{MLP}(\text{LN}(L'))$ . Insieme al pre-norm, è ciò che permette di impilare in profondità il latent transformer.

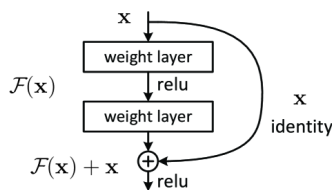


Figura 16: Un blocco residuale: l’input  $x$  “salta” il blocco  $f$  e viene sommato alla sua uscita. Il ramo diretto è una corsia preferenziale per il gradiente.

## 5.4 Softmax — da punteggi a probabilità

La **softmax** trasforma un vettore di numeri reali qualsiasi (i *punteggi*) in una **distribuzione di probabilità**:

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Le sue proprietà: tutti i valori finiscono in  $[0, 1]$ , **sommano a 1**, l’ordine dei punteggi è preservato e i valori più grandi vengono enfatizzati (per via dell’esponenziale).

Nel Perceiver la softmax compare **due volte**: **(1)** nella *mapa di attenzione*, applicata riga per riga — ogni latente ottiene una distribuzione di pesi sui pixel (un “budget di attenzione” che somma a 1); **(2)** alla *fine*, sui logit, per ottenere le probabilità delle classi. Nei calcoli reali si sottrae prima il massimo di ogni riga, per evitare overflow numerico (trucco *log-sum-exp*), senza cambiare il risultato.

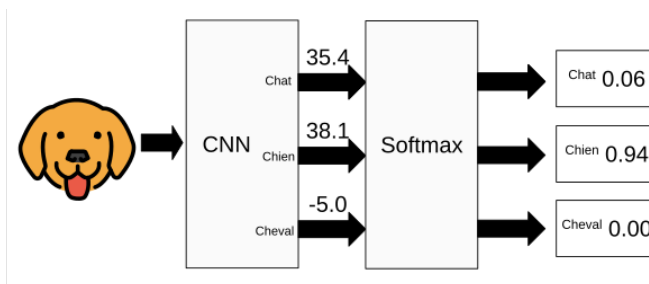


Figura 17: La softmax converte un vettore di punteggi arbitrari in una distribuzione di probabilità: valori non-negativi che sommano a 1.

### Idea chiave

La forza del Perceiver non sta in componenti esotici — usa gli stessi mattoni di un Transformer qualunque — ma in *come* li dispone: cross-attention con latenti, bottleneck, weight sharing. Una volta chiari questi quattro mattoni, ogni formula del forward pass diventa trasparente.

## 6 Dettagli di training (ImageNet)

### In breve

Training su ImageNet: ottimizzatore **LAMB**, 120 epoche, learning rate con *step decay*, **niente dropout**, ~45M parametri (come una ResNet).

- **Ottimizzatore:** LAMB (You et al., 2020). Il paper riporta che il Perceiver è più facile da ottimizzare con LAMB che con Adam.
- **Schedule:** 120 epoche; learning rate iniziale  $4 \times 10^{-3}$  con *step decay* ( $\div 10$  alle epoche 84/102/114). Niente cosine, niente warmup.
- **No dropout:** il paper lo ha provato e ha osservato un *peggioramento* (App. C). Il modello ha già forti pressioni regolarizzanti — bottleneck  $N \ll M$ , weight sharing, weight decay, data augmentation aggressiva (RandAugment, Cubuk et al., 2020).
- **Dimensione:**  $\sim 45M$  parametri, paragonabile a una ResNet usata su ImageNet.

## 7 Risultati chiave

### In breve

**ImageNet 78.0%** su pixel grezzi, senza convoluzioni 2D — alla pari di ResNet-50 e ViT. **Permutation invariance:** 78.0  $\rightarrow$  78.0 sotto permutazione dei pixel, *zero* calo. È il risultato che dimostra la vera generalità del modello.

**Classificazione ImageNet** (top-1, pixel grezzi). Il Perceiver raggiunge **78.0%** lavorando direttamente sui 50 176 pixel *senza convoluzioni 2D*.

Tabella 3: ImageNet top-1 accuracy (validation), Tab. 1 del paper. FF = Fourier features.

| Modello                          | Top-1 (val)  | Note                                        |
|----------------------------------|--------------|---------------------------------------------|
| ResNet-50                        | 77.6%        | convoluzioni 2D                             |
| ViT-B/16                         | 77.9%        | patch 2D                                    |
| Transformer (64 $\times$ 64, FF) | 57.0%        | input downsamplato (costretto da $O(M^2)$ ) |
| <b>Perceiver (FF)</b>            | <b>78.0%</b> | 50 176 pixel grezzi, <b>nessuna conv 2D</b> |

È competitivo con ResNet-50 e ViT-B/16, entrambi con un forte *prior* 2D. Il Transformer puro fallisce (57.0%): la complessità  $O(M^2)$  lo costringe a downsamplare a  $64 \times 64$ , perdendo dettaglio.

**Permutation invariance — il risultato chiave.** I pixel di tutte le immagini vengono mescolati con la *stessa* permutazione fissa. Un modello che codifica la posizione *nell'input* (via Fourier features) dovrebbe esserne immune.

Tabella 4: ImageNet *raw* vs *permuted* (Tab. 2 del paper).

| Modello                               | ImageNet <i>raw</i> | ImageNet <i>permuted</i> |
|---------------------------------------|---------------------|--------------------------|
| <b>Perceiver</b> (Fourier features)   | <b>78.0</b>         | <b>78.0</b>              |
| Perceiver (learned position encoding) | 78.0                | 70.9                     |
| ViT-B/16 (FF)                         | 76.7                | 61.7                     |
| ResNet-50 (FF)                        | 73.5                | 39.4                     |

### Idea chiave — il risultato che “vende” il Perceiver

Il Perceiver con Fourier features è **invariante alla permutazione**: 78.0  $\rightarrow$  78.0, zero calo. La ResNet crolla (73.5  $\rightarrow$  39.4) perché la conv 2D *assume* che i vicini nella griglia siano correlati. Importante: la versione con *learned* position encoding scende a 70.9  $\Rightarrow$  è la scelta delle **Fourier features**, non l'architettura in sé, a dare l'invarianza. Il paper usa questo per argomentare che il Perceiver *non* sfrutta un bias 2D nascosto: è davvero general-purpose.

**Altri domini.** AudioSet (audio+video grezzi):  $\sim 43\%$  mAP. ModelNet-40 (point cloud): 85.7% contro il 91.9% di PointNet++ — il gap si spiega perché il Perceiver usa solo le coordinate  $(x, y, z)$

grezze, senza feature geometriche specializzate. Conferma comunque la generalità dell'architettura.

## 8 Ablation: cosa conta davvero

### In breve

**Latent transformer** essenziale: senza, l'accuracy crolla a  $\leq 45\%$ . **Weight sharing**:  $7\times$  meno parametri e  $+5.1\%$  di accuracy. **Cross-attention interleaved** (alternata) batte "at-start" (tutte all'inizio).

Sintesi degli ablation studies del paper (App. B, Figure 5–6, Tabelle 5–7).

Tabella 5: I tre ablation più importanti: ogni componente, da sola, è essenziale.

| Componente                    | Senza                     | Con (modello completo)       | Conclusione                |
|-------------------------------|---------------------------|------------------------------|----------------------------|
| Latent transformer            | $\leq 45\%$ (o OOM)       | 78.0%                        | il bottleneck è essenziale |
| Weight sharing                | 72.9% (326M)              | 78.0% (45M)                  | regolarizzatore potente    |
| Cross-att. <i>interleaved</i> | 73.7% ( <i>at-start</i> ) | 78.0% ( <i>interleaved</i> ) | rileggere l'input conta    |

- **Il latent bottleneck è essenziale.** Rimuovendo il latent transformer e usando solo cross-attention ripetute, l'accuracy crolla a  $\leq 45\%$  (o va in out-of-memory). Il bottleneck non è solo risparmio computazionale: è il *meccanismo* che integra l'informazione globale tra i latenti.
- **Interleaved > at-start.** 8 cross-attention *alternate* al processo latente  $\rightarrow 78.0\%$ ; le stesse 8 tutte all'inizio  $\rightarrow 73.7\%$ . Il modello deve **ri-leggere** l'input più volte, con latenti progressivamente raffinati.
- **Weight sharing = regolarizzatore.**  $\sim 7\times$  meno parametri,  $+5.1\%$  di accuracy, overfitting eliminato (gap train/val da  $-14.8\%$  a  $-1.5\%$ ).
- **Fourier features.** Più bande di frequenza migliorano le prestazioni **fino alla frequenza di Nyquist**; oltre, nessun guadagno (sono ridondanti).
- $N$ ,  $D$ ,  $\ell$ . Aumentarli aiuta, ma  $D$  oltre un certo punto causa overfitting — mitigato proprio dal weight sharing.

### Da ricordare per l'esame

- **Problema:** la self-attention costa  $O(M^2) \Rightarrow$  **soluzione:** latent bottleneck  $N \ll M$ , costo  $O(MN) + O(N^2)$ .
- **Cross-attention:**  $Q$  dai latenti,  $K/V$  dall'input  $\Rightarrow$  matrice di attenzione *rettangolare*  $N \times M$ . Dim. interna  $d_{QKV} = \min(D, C_{\text{tot}}) = 261$ .
- **Tre stadi:** encoder cross-attention  $\rightarrow$  latent transformer (self-attention, architettura GPT-2)  $\rightarrow$  pooling + classificatore.
- **Forme chiave (ImageNet):**  $X$  è  $50\,176 \times 261$ ,  $L$  è  $512 \times 1024$ , attenzione cross  $512 \times 50\,176$ , attenzione latente  $512 \times 512$ .
- **Fourier PE:** frequenze *linearly spaced* (no NeRF), si *concatena* (no somma) + coordinata grezza  $\Rightarrow$  **permutation invariance**.
- **Weight sharing:** il modello diventa un RNN;  $7\times$  meno parametri,  $+5.1\%$  accuracy.
- **Numeri:** ImageNet 78.0% (Fourier, senza conv 2D); *permuted* 78.0% (invariante); weight sharing 326M  $\rightarrow$  45M parametri, 72.9%  $\rightarrow$  78.0%.
- **Training:** ottimizzatore LAMB, *no dropout*,  $\sim 45$ M parametri.

## Riferimenti

### Paper di riferimento

- Jaegle, A. et al. (2021). *Perceiver: General Perception with Iterative Attention*. ICML 2021. — il paper su cui si basa questo documento.
- Vaswani, A. et al. (2017). *Attention Is All You Need*. NeurIPS. — Transformer e scaled dot-product attention.
- Radford, A. et al. (2019). *Language Models are Unsupervised Multitask Learners*. — architettura GPT-2, usata nel latent transformer.
- Mildenhall, B. et al. (2020). *NeRF*. — Fourier features con frequenze esponenziali (l'alternativa scartata).
- Glorot, X. & Bengio, Y. (2010). — inizializzazione Xavier dei pesi.
- You, Y. et al. (2020). *LAMB*. — ottimizzatore usato per il training su ImageNet.
- Cubuk, E. et al. (2020). *RandAugment*. — data augmentation usata nel training.

### Appendici del paper Perceiver (Jaegle et al., 2021) — mappa rapida

- **App. B** — Ablations: Tabelle 5–7 (weight sharing, cross-attends), Figure 5–6 (effetto di  $N, D, \ell, T$ ).
- **App. C** — Dettagli architetturali: LayerNorm pre-norm, cross-attention single-head,  $d_{QKV} = 261$ , no bias in  $Q/K/V$ , no dropout, init latenti  $\mathcal{N}(0, 0.02^2)$ .
- **App. D** — Positional encoding: frequenze linearly spaced tra 1 e  $\mu/2$ , confronto esplicito con NeRF.
- **App. F** — Modifiche rispetto alla prima versione: rimozione del dropout, pooling *prima* della proiezione.